Nombre:	Luis de la Garza González
Matrícula:	al03101869
Nombre del curso:	Aprendizaje Profundo.
Trabajo:	Fase II.- Ejecución.
Nombre del profesor:	Rodrigo Francisco Román Godínez
Fecha:	08.03.2026

Contenido

Objetivo 3

Instrucciones 3

Desarrollo 4

1. Identificación de la tarea a resolver y obtención los datos de entrenamiento y de prueba; instalación y configuración de las librerías a utilizar 4

2. Separa los conjuntos de datos obtenidos en tres subconjuntos: entrenamiento, validación y prueba. .. 5

3. Preprocesa los datos según se requiera, ya sea estandarizando medidas, escala de grises, formato, cantidad de pixeles y codifica la categoría en donde correspondan. 6

4. Construye una red neuronal convolucional para realizar la clasificación de imágenes. 7

5. Evalúa los resultados que se obtuvieron con el clasificador construido. 10

5.1 Evaluación en el conjunto de prueba 10

5.2 Curvas de entrenamiento 10

5.3 Matriz de confusión para evaluar el clasificador 11

Conclusiones 14

Liga al Código 15

Objetivo

Aplicar el aprendizaje profundo en la resolución de una problemática de clasificación de imágenes.

Instrucciones

La clasificación de imágenes es una de las actividades más importantes en el campo de la visión computacional. Es una tarea donde el objetivo es asignar una o varias etiquetas (o categorías) a una imagen digitalizada de forma automática.

Deberás plantear un problema de clasificación de imágenes, así como su solución, utilizando redes neuronales convolucionales. La tarea puede ser clasificar animales, como perros o gatos, o diferentes tipos de aves, o incluso algún padecimiento médico.

La solución debe utilizar Keras y TensorFlow en Python. Se sugiere que consideres las siguientes fuentes:

- Abu, M., Indra, N., Rahman, A., Sapiee, N., y Ahmad, I. (2019). A study on Image Classification based on Deep Learning and Tensorflow. International Journal of Engineering Research and Technology, 12(4). Recuperado de https://www.ripublication.com/irph/ijert19/ijertv12n4_16.pdf
- Brownlee, J. (2019). How to Classify Photos of Dogs and Cats (with 97% accuracy). Recuperado de https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-to-classify-photos-of-dogs-and-cats/
- TensorFlow. (s.f.). Clasificación de imágenes. Recuperado de https://www.tensorflow.org/tutorials/images/classification

Sigue las instrucciones para llevar a cabo el proyecto:

1. Identifica la tarea a resolver y obtén los datos de entrenamiento y de prueba, así como la instalación y configuración de las librerías que se utilizarán.
2. Separa los conjuntos de datos obtenidos en tres subconjuntos: entrenamiento, validación y prueba.
3. Preprocesa los datos según se requiera, ya sea estandarizando medidas, escala de grises, formato, cantidad de pixeles y codifica la categoría en donde correspondan.
4. Construye una red neuronal convolucional para realizar la clasificación de imágenes.
5. Evalúa los resultados que se obtuvieron con el clasificador construido.
6. Presenta tus conclusiones.

Desarrollo

1. Identificación de la tarea a resolver y obtención los datos de entrenamiento y de prueba; instalación y configuración de las librerías a utilizar

De la página Web kaggle.com, vamos a bajar el dataset Animals-10, el cual cuenta con imágenes organizadas en 10 clases diferentes, de éstas solamente vamos a utilizar 5. El formato de las imágenes es JPG. Las imágenes vienen agruparas en directorios conteniendo estos el nombre de la especie (en italiano)

Debido a que la resolución de las imágenes es relativamente alta y de que no cuento con la potencia de cómputo necesaria, vamos a reducir las imágenes a 128 x 128 pixeles.

Instalación y configuración de las librerías a utilizar:

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, regularizers
import matplotlib.pyplot as plt
import os
```

El siguiente código muestra la obtención los datos de entrenamiento y de prueba, los cuales están ubicados en el directorio raw-img en Drive:

```
from google.colab import drive
drive.mount('/content/drive')

data_dir = "/content/drive/MyDrive/raw-img"
img_size = (128, 128)
batch_size = 32
```

2. Separa los conjuntos de datos obtenidos en tres subconjuntos: entrenamiento, validación y prueba.

Usaremos image_dataset_from_directory para crear un dataset y luego lo separamos en tres subconjuntos: entrenamiento, validación y prueba.

```
dataset = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    image_size=img_size,
    batch_size=batch_size,
    shuffle=True
)

class_names = dataset.class_names
num_classes = len(class_names)
print("Clases:", class_names)
```

```
…
Found 14142 files belonging to 5 classes.
Clases: ['cane', 'cavallo', 'elefante', 'farfalla', 'gallina']
```

Haremos una partición aproximada 80% train, 10% val, 10% test:

```
dataset_size = 0
for _ in dataset:
    dataset_size += 1

train_size = int(0.8 * dataset_size)
val_size   = int(0.1 * dataset_size)
test_size  = dataset_size - train_size - val_size

train_ds = dataset.take(train_size)
rest = dataset.skip(train_size)
val_ds = rest.take(val_size)
test_ds = rest.skip(val_size)

print("Batches - train:", train_size, "val:", val_size, "test:", test_size)
```

```
…
Batches - train: 353 val: 44 test: 45
```

3. Preprocesa los datos según se requiera, ya sea estandarizando medidas, escala de grises, formato, cantidad de pixeles y codifica la categoría en donde correspondan.

Preprocesamiento de datos:

- Reescalamos píxeles a [0,1].
- Opcional: data augmentation ligero.
- Las etiquetas ya vienen codificadas como enteros; la capa final usará SparseCategoricalCrossentropy.

```
normalization_layer = layers.Rescaling(1./255)

data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

AUTOTUNE = tf.data.AUTOTUNE

def prepare(ds, augment=False):
    ds = ds.map(lambda x, y: (normalization_layer(x), y),
                num_parallel_calls=AUTOTUNE)
    if augment:
        ds = ds.map(lambda x, y: (data_augmentation(x, training=True), y),
                    num_parallel_calls=AUTOTUNE)
    return ds.cache().prefetch(AUTOTUNE)

train_ds_prep = prepare(train_ds, augment=True)
val_ds_prep   = prepare(val_ds, augment=False)
test_ds_prep  = prepare(test_ds, augment=False)
```

4. Construye una red neuronal convolucional para realizar la clasificación de imágenes.

```
model = models.Sequential([
    layers.Input(shape=(128, 128, 3)),

    # Bloque 1
    layers.Conv2D(32, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    # Bloque 2
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    # Bloque 3
    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    # Bloque 4 (más profundo para 128x128)
    layers.Conv2D(256, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(256, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(),

    # Clasificación
    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu',
kernel_regularizer=regularizers.l2(1e-4)),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])

model.summary()
```

…

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 32)	896
batch_normalization (BatchNormalization)	(None, 128, 128, 32)	128
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 128, 128, 32)	128
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 64, 64, 64)	256
conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 64, 64, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 32, 32, 128)	512
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 32, 32, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0
conv2d_6 (Conv2D)	(None, 16, 16, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 16, 16, 256)	1,024
conv2d_7 (Conv2D)	(None, 16, 16, 256)	590,080

batch_normalization_7 (BatchNormalization)	(None, 16, 16, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 256)	65,792
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 5)	1,285

Total params: 1,243,173 (4.74 MB)

Trainable params: 1,241,253 (4.74 MB)

Non-trainable params: 1,920 (7.50 KB)

Compilación:

```
model.compile(
   optimizer=keras.optimizers.Adam(learning_rate=1e-4),
   loss=keras.losses.SparseCategoricalCrossentropy(),
   metrics=['accuracy']
)
```

Entrenamiento:

Debido al escaso poder de cómputo del que disponga, usaremos solamente 10 épocas.

```
epochs = 10

history = model.fit(
    train_ds_prep,
    validation_data=val_ds_prep,
    epochs=epochs
)
```

```
…
Epoch 1/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2246s 6s/step - accuracy: 0.4470 - loss: 1.4833 -
val_accuracy: 0.1811 - val_loss: 2.9755
Epoch 2/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2092s 6s/step - accuracy: 0.6237 - loss: 1.0016 -
val_accuracy: 0.3913 - val_loss: 1.8721
Epoch 3/10
```

```
353/353 ━━━━━━━━━━━━━━━━━━━━ 2110s 6s/step - accuracy: 0.6987 - loss: 0.8430 -
val_accuracy: 0.4283 - val_loss: 1.8212
Epoch 4/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2078s 6s/step - accuracy: 0.7347 - loss: 0.7355 -
val_accuracy: 0.4567 - val_loss: 1.7177
Epoch 5/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2075s 6s/step - accuracy: 0.7724 - loss: 0.6656 -
val_accuracy: 0.4453 - val_loss: 1.7998
Epoch 6/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2011s 6s/step - accuracy: 0.7988 - loss: 0.5864 -
val_accuracy: 0.4517 - val_loss: 1.8081
Epoch 7/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2009s 6s/step - accuracy: 0.8211 - loss: 0.5217 -
val_accuracy: 0.4432 - val_loss: 1.9154
Epoch 8/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2008s 6s/step - accuracy: 0.8480 - loss: 0.4607 -
val_accuracy: 0.5199 - val_loss: 1.5109
Epoch 9/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2007s 6s/step - accuracy: 0.8797 - loss: 0.3851 -
val_accuracy: 0.4993 - val_loss: 1.8701
Epoch 10/10
353/353 ━━━━━━━━━━━━━━━━━━━━ 2006s 6s/step - accuracy: 0.8978 - loss: 0.3240 -
val_accuracy: 0.5689 - val_loss: 1.3774
```

5. Evalúa los resultados que se obtuvieron con el clasificador construido.

5.1 Evaluación en el conjunto de prueba

```
test_loss, test_acc = model.evaluate(test_ds_prep)
print(f"Loss de prueba: {test_loss:.4f}")
print(f"Accuracy de prueba: {test_acc:.4f}")
```

```
…
45/45 ━━━━━━━━━━━━━━━━━━━━ 117s 1s/step - accuracy: 0.5742 - loss: 1.3333
Loss de prueba: 1.3554
Accuracy de prueba: 0.5723
```

5.2 Curvas de entrenamiento

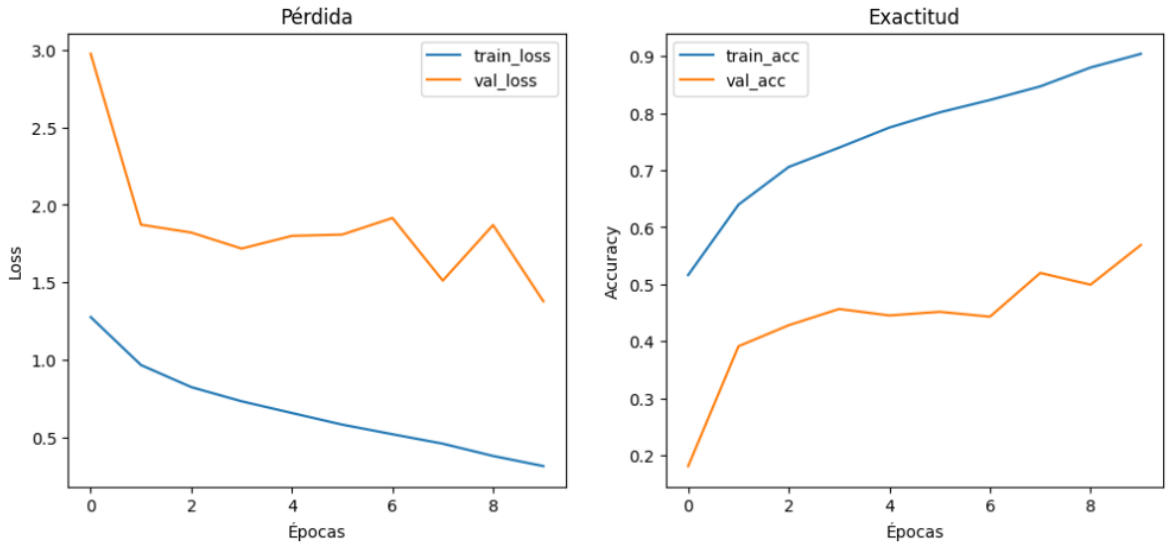
```
plt.figure(figsize=(12,5))

# Pérdida
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.title("Pérdida")
plt.xlabel("Épocas")
```

```
plt.ylabel("Loss")
plt.legend()

# Exactitud
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='train_acc')
plt.plot(history.history['val_accuracy'], label='val_acc')
plt.title("Exactitud")
plt.xlabel("Épocas")
plt.ylabel("Accuracy")
plt.legend()

plt.show()
```



Podemos ver que el modelo va mejorando la precisión con el paso de las épocas, sin embargo vamos a parar en 10.

5.3 Matriz de confusión para evaluar el clasificador

```
import numpy as np

y_true = []
y_pred = []

for images, labels in test_ds_prep:
    preds = model.predict(images)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))
```

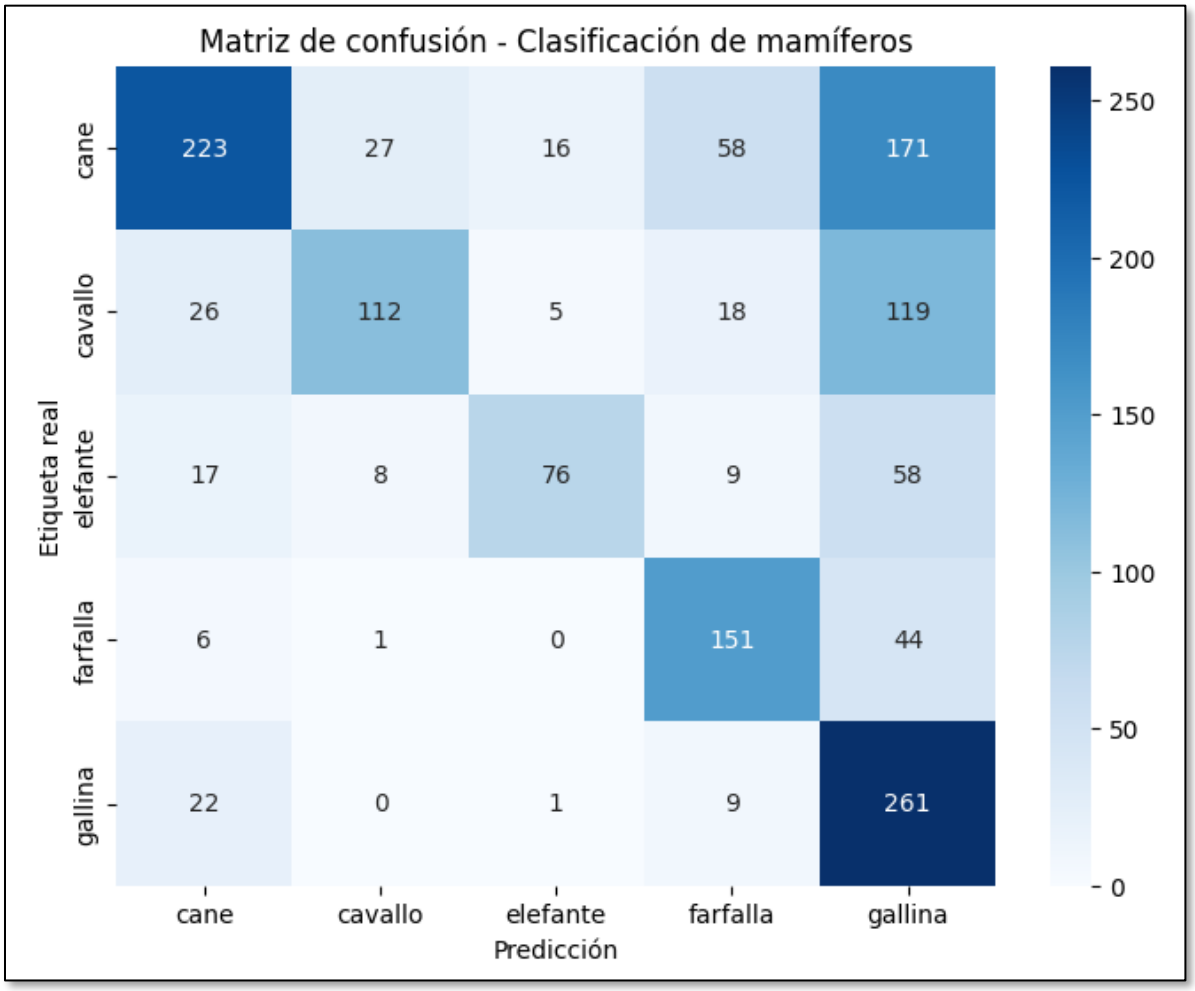
```
y_true = np.array(y_true)
y_pred = np.array(y_pred)

# Construir la matriz de confusión
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true, y_pred)
cm

# Visualizar la matriz de confusión
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names,
            yticklabels=class_names)
plt.xlabel("Predicción")
plt.ylabel("Etiqueta real")
plt.title("Matriz de confusión - Clasificación de mamíferos")
plt.show()
```

```
…
array([[223,  27,  16,  58, 171],
       [ 26, 112,   5,  18, 119],
       [ 17,   8,  76,   9,  58],
       [  6,   1,   0, 151,  44],
       [ 22,   0,   1,   9, 261]])
```



Podemos ver mayor precisión en la clasificación de las categorías perro, caballo y elefante; y menor precisión sobre todo en la gallina.

Conclusiones

En esta actividad se aplicaron técnicas de “aprendizaje profundo” para resolver un problema de “clasificación de imágenes multiclase”, utilizando una “red neuronal convolucional (CNN)” implementada con “TensorFlow y Keras”. El proyecto permitió recorrer de forma integral el flujo de trabajo típico en visión computacional, desde la preparación y preprocesamiento de los datos hasta la evaluación final del modelo entrenado.

El uso de un conjunto de datos con “10 clases de animales” y alrededor de “28,000 imágenes” (para el ejercicio se utilizaron solamente 5 de las 10 clases) representó un reto significativo, especialmente considerando la reducción de resolución de las imágenes a “128×128 píxeles” por limitaciones de cómputo. Esta decisión permitió entrenar el modelo de forma viable, aunque también impactó negativamente en la capacidad del modelo para capturar detalles finos necesarios para distinguir entre clases visualmente similares.

Durante el entrenamiento, el modelo mostró una “alta precisión en el conjunto de entrenamiento”, alcanzando valores superiores al 95%, mientras que el desempeño en los conjuntos de validación y prueba fue considerablemente menor, con una “exactitud de 57.23% en el conjunto de prueba”. Es posible que este comportamiento evidencie un “problema de sobreajuste”, donde el modelo aprende bien los datos de entrenamiento pero no generaliza adecuadamente a datos no vistos.

Las curvas de entrenamiento y la matriz de confusión refuerzan este diagnóstico, mostrando que, aunque la red logra aprender representaciones internas complejas, no consigue discriminar correctamente entre todas las clases. Esto es esperable dada la alta cantidad de categorías, la similitud visual entre especies y la ausencia de técnicas más avanzadas como “transfer learning” o arquitecturas más profundas y especializadas.

A pesar de estas limitaciones, el proyecto cumple con el objetivo de “aplicar y comprender los fundamentos de las CNN en un problema realista”, demostrando la importancia del preprocesamiento, la arquitectura del modelo y la evaluación crítica de resultados.

Como mejoras al ejercicio se pueden explorar “modelos preentrenados”, ajustar hiperparámetros, aumentar la resolución de las imágenes y aplicar técnicas de regularización adicionales para mejorar la capacidad de generalización del clasificador, lo anterior contando con un equipo de cómputo con mayor potencia.

Liga al Código

https://github.com/luisgg121/DL-Actividad-Fase-II.git

Archivo: DL-Fase-II.ipynb